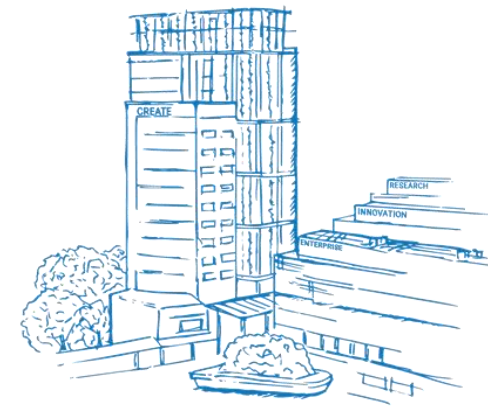


Agentic AI Workshop

A hands-on six-week series for researchers: from natural-language interfaces to autonomous scientific agents.

Dr. Kerem Delikoyun · TUMCREATE · 20 May 2026



WEEK 1

Foundations of Gen AI: What we'll cover

Look ma no hands: Now, we can talk to our machines and they are pretty smart too!

1 · How we got here

From punch cards to natural language as a programming, the abstraction layer is changing.

2 · CLI-native coding agents

Gentle intro to Claude Code, and the files that make it think: `CLAUDE.md`, `SOUL.md`, skills, hooks, MCP.

3 · Organon

An agentic OS for scientists, built on Claude Code primitives.

4 · Live demo

From `/lets-go` to writing your next article — live.

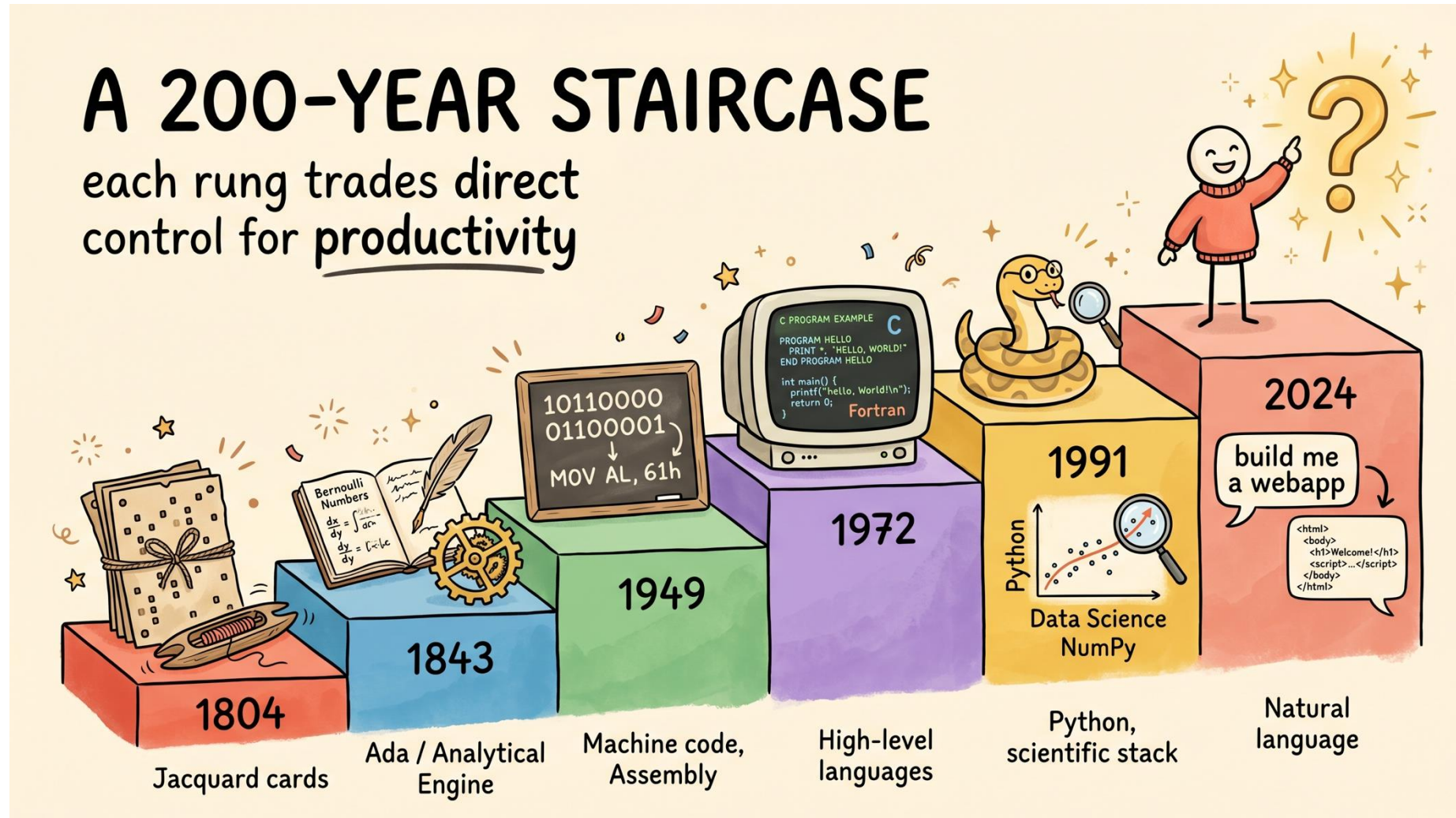
You'll leave something to break tonight, in a good way...

PART 1

Two long paths converge...

For two hundred years, we've been climbing toward easier ways to talk to machines: punch cards, assembly, compilers and programming languages. For over eighty years, machines have been learning to understand us: perceptrons, deep learning, large language models. The two stories meet at *natural language*.

The 200-year of abstraction staircase



Each rung trades direct control for productivity. We just climbed the next step.

1804 · The first program lived in a card

Joseph Marie Jacquard, Lyon. A loom controlled by **punch cards**.

Each card row selects which warp threads lift.

A chain of cards weaves an image: a portrait, a brocade, a silk pattern: *without changing/breaking the machine.*

*The **program** has just been separated from the **machine**.*

Why this is the start

- The same loom runs any pattern by swapping cards.
- The pattern is **stored** — reproducible, shareable, editable.
- The Bureau of the Census buys descendants of this idea in 1890. IBM is built on it.



A chain of punch cards mounted on a Jacquard loom: the first programmable machine.

Photo: Wikimedia Commons

1843 · Ada saw what the machine could become

Ada Lovelace translates Menabrea's 1842 paper on Babbage's **Analytical Engine** and adds her own *Notes A–G*.

Note G — the **first published algorithm intended for machine execution** (Bernoulli numbers).

"The Analytical Engine... can do whatever we know how to order it to perform." — Note G

In *Note A*, she sees the engine could manipulate **symbols, not just numbers**: even compose music.

Software predates working hardware by a century.



Ada Lovelace · M. S. Carpenter, 1836 (PD).

1940s–50s · The human stopped writing bits

Machine code

What the CPU executes.

```
10110000 01100001  
00000100 00000010  
11110100 00000000
```

Assembly · 1947 → 1949

Mnemonic 1:1 with the same op-codes.

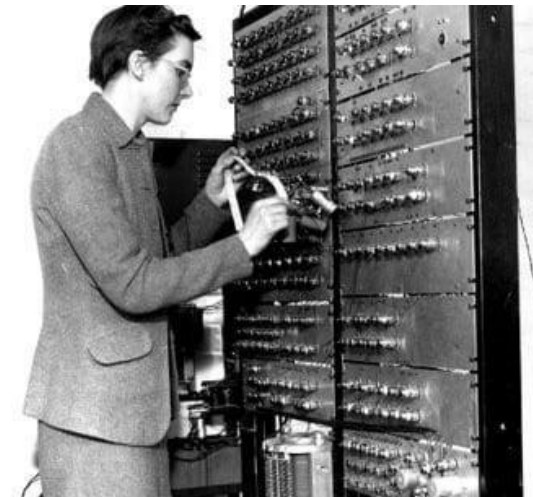
```
MOV AL, 61h  
ADD AL, 02h  
HLT
```

*Every layer above this exists for the same reason:
human working memory is small.*

`MOV` doesn't help the CPU. It helps the human to keep track.

The op-codes did not change.

The **person describing them** did.



Kathleen Booth

1957–1972 · The compiler hides the CPU

- 1957** **FORTRAN** — numerical computing for scientists.
First widely-used compiled language. *Backus, IBM.*
- 1958** **LISP** — symbolic AI, lambda calculus, garbage collection
Powered the MIT AI Lab for thirty years. *McCarthy.*
- 1959** **COBOL** — business data. Still running banks in 2026!
CODASYL committee, building on Hopper's FLOW-MATIC.
- 1972** **C** — portable systems. Unix is rewritten in it in 1973.
The Linux kernel under your laptop is its grandchild.
Ritchie, Bell Labs.

SAME IDEA, FOUR FLAVOURS

```
DO 10 I = 1, N  
  S = S + A(I)  
10 CONTINUE
```

```
(defun fact (n)  
  (if (= n 0) 1  
      (* n (fact (- n 1)))))
```

```
PROCEDURE DIVISION.  
  ADD AMOUNT TO TOTAL.
```

```
for (int i = 0; i < n; i++)  
  sum += a[i];
```

The compiler is a translator. You write what you want; it picks the registers. **One rung up.**

1991 → today · Python ate scientific computing

Guido van Rossum, 1991: *readability over cleverness*.

Then a 20-year build-out of scientific libraries:

- **SciPy** · 2001 · numerical methods
- **NumPy 1.0** · 2006 · arrays
- **pandas** · 2009 · dataframes
- **scikit-learn** · 2010 · classical ML
- **TensorFlow** · 2015 · deep learning
- **PyTorch** · 2017 · research-first DL

WHAT IT LOOKS LIKE

```
U, S, Vt = np.linalg.svd(A)
```

One line. Hundreds of **LAPACK** Fortran routines underneath.

You do **not** read the implementation.

You trust the **interface**.

Compounds over every abstraction layer.



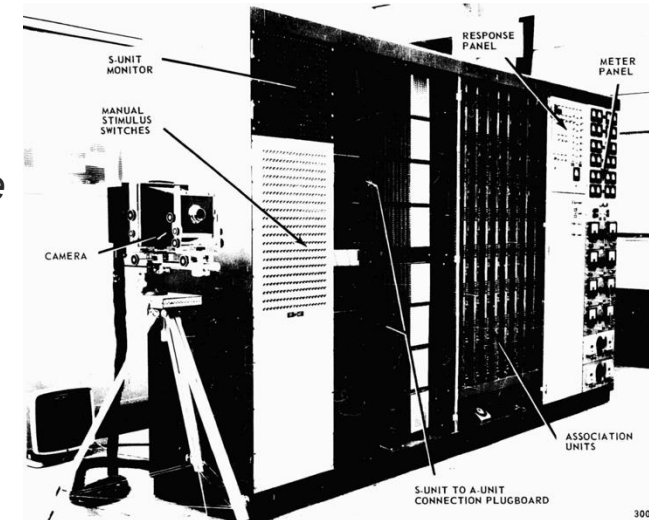
PARALLEL UNIVERSE

Meanwhile, lot was happening with AI

The abstraction story made us easier to write down. The AI story made machines smarter in language, in vision, in audio, and in the spaces between. By 2024, modern models can read, see, listen, and reason across all of it at once.

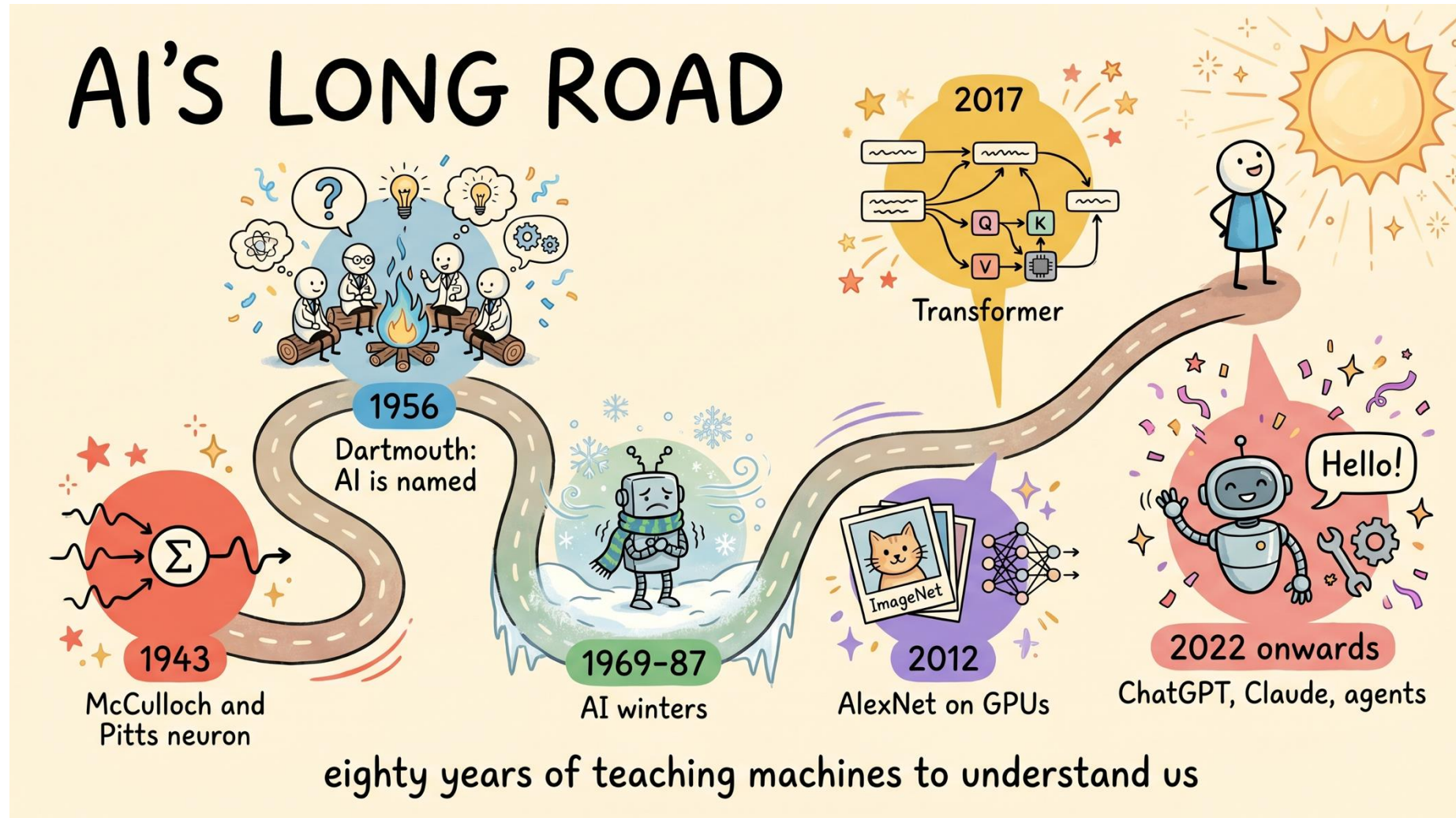
AI's long road · the milestones

- **1943 · McCulloch & Pitts** — the artificial neuron, on paper.
- **1950 · Turing** — "*Can machines think?*" The Imitation Game.
- **1956 · Dartmouth** — McCarthy & co. coin *artificial intelligence*
- **1957–58 · Rosenblatt's Perceptron** — algorithm + Mark I hardware. NYT calls it the embryo of an electronic brain.
- **1973–93 · Two AI winters** — Lighthill (1973),
Roots: Minsky & Papert on XOR (1969) → 1st AI winter
LISP-machine collapse (1987–93) → 2nd AI winter
- **1986 · Backpropagation** — Rumelhart, Hinton, Williams.
Math: Linnainmaa, 1970.



Mark I Perceptron · Rosenblatt, Cornell, 1958. Operator's manual / Wikimedia (PD).

AI's long road · the parallel universe



Eighty years of teaching machines to understand us. The road met the staircase in 2024.

AI's long road · the breakthroughs

- 1997** **Deep Blue** beats Kasparov, 3.5–2.5. Symbolic search + hand-tuned heuristics.
- 2012** **AlexNet**: Krizhevsky, Sutskever, Hinton win *ImageNet* by a 10-point margin. GPUs change everything.
- 2017** **Transformer**: *Attention Is All You Need*. Vaswani et al. (Google Brain / Google Research / U. Toronto). The architecture under every modern LLM.
- 2020** **GPT-3**: 175 B parameters. Few-shot learning works. Scaling laws hold.
- Nov 2022** **ChatGPT**: 100 M users in two months. Natural-language interfaces leave the lab.
- 2024+** **Agentic AI**: Claude, GPT-4o, Gemini gain tool use, memory, code execution. Models start *doing things*, not just *answering*...

2024 · English becomes the programming language

What you write

Write me the Python code
to find all the prime numbers
less than 1000.

What the AI writes — and runs

```
def primes_below(n):  
    sieve = [True] * n  
    sieve[0] = sieve[1] = False  
    for i in range(2, int(n**0.5) + 1):  
        if sieve[i]:  
            sieve[i*i::i] = [False] * len(sieve[i*i::i])  
    return [i for i, is_prime in enumerate(sieve) if is_prime]  
  
print(primes_below(1000))
```

What you write/how to interact with machines changes: you describe **intent/plan/goal**; the AI agent writes the code, runs the tests, reads the output, and reports back.

And over time it learns *how you write, what you prefer, what you don't need to say twice.*

The new contract is the test suite

You never read CPython source before calling

```
numpy.dot
```

You read the **docs**, trust the **tests**.

The same deal scales to the natural-language layer:

- Describe the **intent** in English.
- Write the **tests** that define correctness.
- The agent writes the code.
- Tests pass $\Rightarrow$ you ship.

THE CONTRACT, FORMALISED

```
def test_save_idea_persists(client, db):  
    r = client.post("/ideas",  
                    json={"text": "hello"})  
    assert r.status_code == 201  
    assert db.count("ideas") == 1
```

The test is the **specification**. The code is **disposable**.

Reframed from Erik Schultntz's talk from Anthropic · YT/fHWFF_pnqDk

This is the new abstraction and verification layer

An LLM call is a **function**:

```
input → model → output
```

The way scientists think is **not** a function. It's a workflow:

- **memory**, across sessions
- **identity**, a consistent collaborator
- **tools**, literature, statistics, plotting, writing
- **learnings**, sharper next week than this week

AGENT = LLM + FOUR PRIMITIVES

The next two sections show what those primitives look like as **real files on disk**

first in **Claude Code**: the CLI-native coding agent for the natural-language layer.

then in **Organon**, a scientific workflow built on top.

This is the bridge from *building an app* to *building a colleague*.

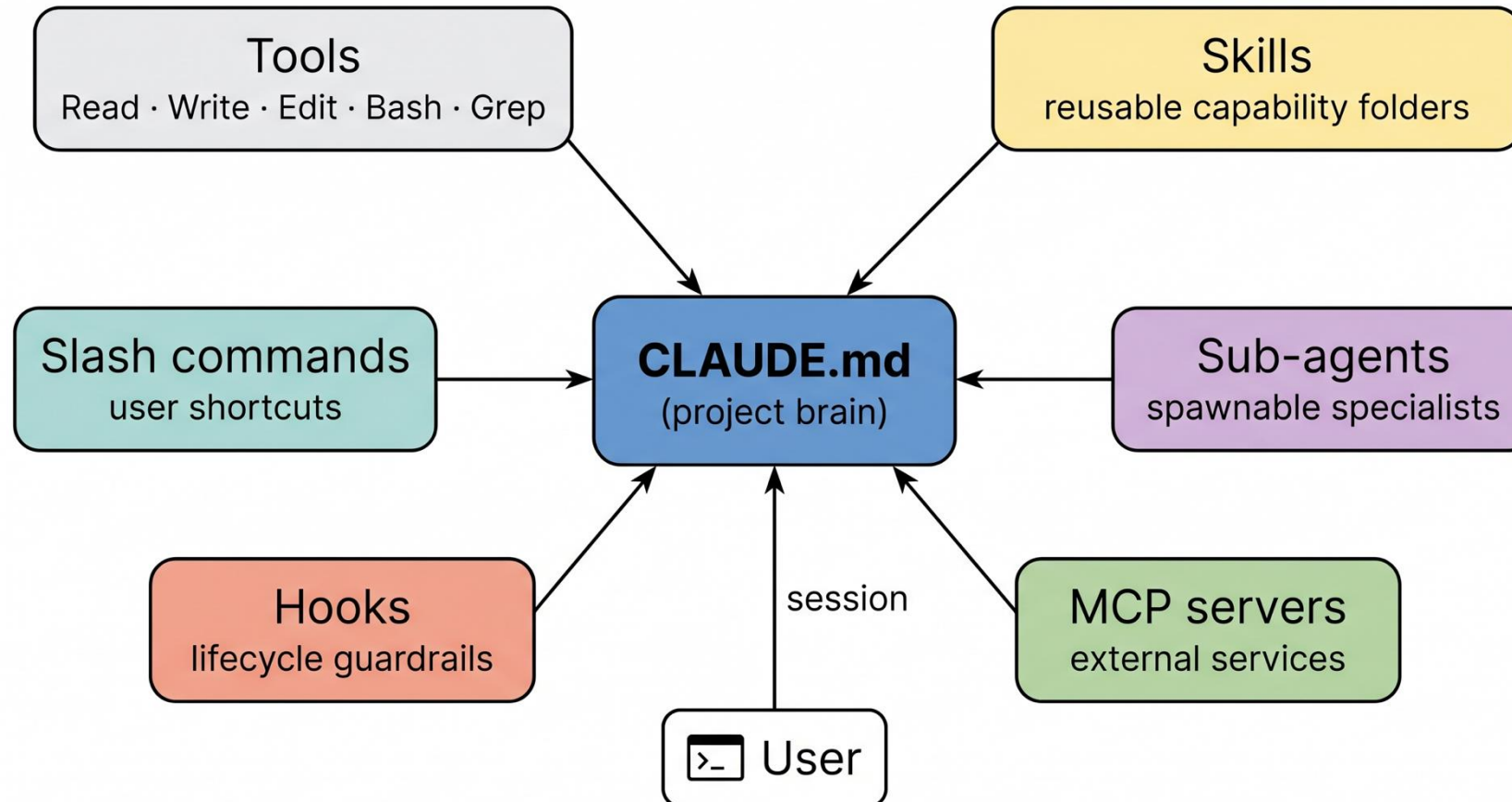
PART 2

CLI-based AI agents for “vibe” coding

Terminal-native, agentic coding agents. Not an autocomplete.

Not a chatbot. A colleague that reads your repo, runs your tools, and improves itself.

Claude Code · the primitives at a glance



One brain/manual (CLAUDE.md), six primitives.

What Claude Code is · mental model

Three things at once

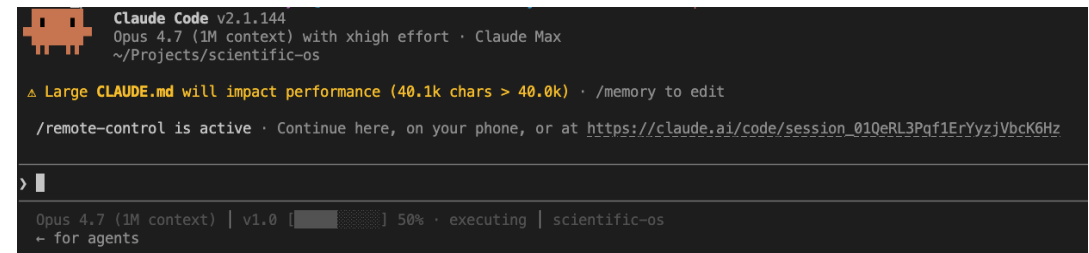
- A **CLI process** you run from any project directory
- A **conversation** with Claude that has *eyes* (file reads), *hands* (tool use), and *memory* (memory.md)
- An **extensible runtime**: skills, sub-agents, hooks, MCP servers

What it is not

- An autocomplete (Copilot)
- A web chatbot (ChatGPT in your browser)
- A black box: everything is files on disk you can read and edit

```
# Install once
$ npm install -g @anthropic-ai/claude-code

# Run anywhere in a project
$ cd projects/my-research
$ claude
```



```
Claude Code v2.1.144
Opus 4.7 (1M context) with xhigh effort · Claude Max
~/Projects/scientific-os

⚠ Large CLAUDE.md will impact performance (40.1k chars > 40.0k) · /memory to edit

/remote-control is active · Continue here, on your phone, or at https://claude.ai/code/session_010eRL3Pqf1ErYyzjVbcK6Hz

> █

Opus 4.7 (1M context) | v1.0 [██████████] 50% · executing | scientific-os
~ for agents
```

The five primitives

1 · CLAUDE.md

The project's brain. Loaded every session. Says what the codebase is, what conventions hold, what *not* to do.

2 · Tools

Read , Write , Edit , Bash , Grep , Glob , WebFetch , WebSearch . The hands.

3 · Skills

Folders under `.claude/skills/{name}/` — reusable capabilities with their own triggers, references, scripts.

4 · Sub-agents

Spawnable specialists (Explore , Plan , domain agents). Run in parallel, write their own files, report back.

5 · MCP servers

Model Context Protocol. Plug external services (Supabase, GitHub, custom APIs) into the same tool surface.

PLUS

Hooks — shell commands that fire on PreToolUse , PostToolUse , SessionStart .

Slash commands — keystroke shortcuts you author (/lets-go , /wrap-up).

CLAUDE.md · the project's brain

```
# CLAUDE.md (excerpt from Organon)
```

Heartbeat

Run at the start of every session:

1. Load **SOUL.md**, **USER.md**, today's memory
2. Load **research-profile.md**
3. Scan available skills
4. Run **/lets-go** automatically

Skill Routing

1. System ops first (table below)
2. Match user phrasing to Skill Registry triggers
3. Cascade: adjacent skill → ToolUniverse → web

Conventions

- Python: snake_case, PEP 8, type hints
- Shell: **set -euo pipefail**
- Outputs land in **projects/{category}/**

What this buys you

- **No re-onboarding** — every session inherits the rules
- **Versioned norms** — conventions live in git
- **Routing for free** — "search papers" becomes the right skill, every time
- **Multi-collaborator** — new humans *and* new AI sessions read the same constitution

*CLAUDE.md is not docs about your codebase. It's the codebase's **operating manual** that the agent reads first.*

Tools · the agent's hands

Built-in

- `Read` · file contents (markdown, PDFs, notebooks, images)
- `Write` · new files
- `Edit` · exact-string replacements
- `Bash` · shell commands (sandboxed via deny rules)
- `Grep` · ripgrep across the repo
- `Glob` · pattern-based file discovery
- `WebFetch` / `WebSearch` · the open web

Permission model

Every tool call is gated by `.claude/settings.json` —
allow / ask / deny lists with regex matching.

A TYPICAL STEP

You: "Why is /api/ideas slow?"

Claude (Read): `app/api/ideas/route.ts`

Claude (Grep): `"SELECT * FROM ideas"`

Claude (Read): `lib/db.ts`

Claude (Bash): `psql -c "EXPLAIN ..."`

Claude → You: "No index on user_id.

Migration drafted at
supabase/migrations/
0042_user_id_idx.sql"

You read the result. You approve. You move on.

Skills · reusable/extendable capabilities

A skill is a **folder with a contract**:

```
.claude/skills/sci-literature-research/  
├── SKILL.md      ← YAML + how-to  
├── references/   ← deep material  
│   ├── pubmed.md  
│   └── citation-styles.md  
├── scripts/     ← runnable helpers  
│   └── search.py  
└── assets/      ← templates
```

The YAML names triggers (*"search papers"*, *"PubMed"*) and negative triggers (*"not a writing skill"*).

When you say "find me five recent papers on cancer", Claude **routes to this skill silently**.

WHY FOLDERS, NOT PROMPTS

- **Composable** — one skill can call another
- **Versionable** — under git, with tests
- **Shareable** — copy the folder, the capability moves with it
- **Inspectable** — you can read what the agent will do *before* it does it

Organon ships with **30+ skills** across science, viz, ops, tools, and meta categories.

Sub-agents · parallel specialists

The main Claude session can **spawn smaller Claude sessions** with narrower tool sets and isolated context:

- `Explore` — fast read-only repo search
- `Plan` — design implementation strategy
- `general-purpose` — open-ended research
- *custom* — project-specific specialists

Each one writes its own files, returns a summary, and **does not pollute the main context**.

Independent work runs **in parallel** — one call dispatches many.

```
# Inside Organon, one user request:  
> brief the literature on GLP-1 weight regain  
  
[main] dispatches three sub-agents:  
→ Explore: search local notes  
→ sci-literature-research: PubMed  
→ sci-trending-research: Reddit + X  
  
# 90 seconds later, all three return.  
# Main writes the synthesis brief.
```

MCP · plug external services in

Model Context Protocol — open standard (Anthropic, Nov 2024; donated to the Linux Foundation's Agentic AI Foundation, Dec 2025) for exposing external services as tools to any LLM client.

In Claude Code, you list MCP servers in `.mcp.json` :

```
{
  "paperclip": {
    "url": "https://paperclip.gxl.ai/mcp"
  },
  "supabase": {
    "command": "npx",
    "args": ["-y", "@supabase/mcp"]
  }
}
```

The agent now has *new tools* — `paperclip__search`, `supabase__query`, etc.

WHY THIS MATTERS

- One protocol — works with Claude, Cursor, Goose, Continue, ...
- **Vendor-neutral** — no lock-in to a single agent
- **Auth handled per server** — OAuth, API keys, token refresh
- **Discoverable** — `tools/list` returns the schema

If you have a domain API (lab instrument, internal database, EHR connector), wrap it in MCP and **every agent in your org can use it.**

Hooks · automated guardrails

Hooks are **shell commands** the harness runs at lifecycle events:

Event	Fires
SessionStart	new session boots
PreToolUse	before any tool call
PostToolUse	after a tool call returns
UserPromptSubmit	each user message
Stop	session ends

Hooks can **block** a tool call by returning non-zero — the agent sees the error and adapts.

```
# .claude/settings.json (excerpt)
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Write|Edit|MultiEdit",
      "command": "scripts/verify_gate.py"
    }],
    "SessionStart": [{
      "command": "echo 'Run heartbeat'"
    }]
  }
}

# Organon uses PreToolUse to refuse
# writes to .bib files that fail
# citation-verify checks.
```

Slash commands · your own shortcuts

Author your own commands as markdown files under

`.claude/commands/{name}.md` :

```
.claude/commands/  
├── lets-go.md    ← daily heartbeat  
├── wrap-up.md    ← session close  
├── archive-gsd.md ← project archive  
└── ultrareview.md ← multi-agent review
```

Each is a **prompt template** Claude executes when you type `/lets-go` .

Skills also expose slash commands automatically. The two systems blur on purpose.

REAL ORGANON COMMANDS

- `/lets-go` — load identity + research context, recap yesterday, propose today's first action
- `/wrap-up` — review deliverables, log learnings, commit
- `/gsd:plan-phase` — structured phase planning
- `/gsd:debug` — scientific-method debugging session

Each one is a **few-hundred-word markdown file** you can read, edit, version, fork.

Putting it together - one session

```
$ cd ~/projects/my-research && claude
// SessionStart hook fires → runs /lets-go automatically
[heartbeat] Loading SOUL.md, USER.md, today's memory, last 10 learnings...
[heartbeat] Research profile: computational biology, GLP-1 cohort active
[heartbeat] Skills available: 33 MCPs connected: paperclip, supabase, gdrive
[heartbeat] Last session: started writeup on weight-regain mechanisms. Open thread: cohort N=87 enrolled.
> draft the introduction section using the literature I pulled yesterday
[routing] sci-writing — matched trigger "draft the introduction"
[reading] projects/glp1-weight-regain/literature/ (12 papers)
[reading] research_context/research-profile.md (writing style)
[reading] context/learnings.md → sci-writing section (your past feedback)
[reading] context/memory/2026-05-14.md (yesterday's session)
[writing] projects/glp1-weight-regain/manuscripts/intro_draft.md
[hook ] PreToolUse:Edit → citation_verify.py → 12 / 12 OK
[done ] Draft saved. Ran humanizer (standard mode, score 8.4/10). Auto-opened in Cursor.
```

PART 3

Organon

An agentic operating system for scientists, built on Claude Code.

Spend less time on the boring parts of research, more on the exciting discoveries.

Why an OS for scientists

A scientist's time is fragmented:

- days — finding recent papers
- days/weeks — data cleaning/analysis, statistical tests
- days/weeks — drafting an article
- days/weeks — making a figure presentable
- days/weeks — formatting citations

Organon has tools and skills for each.

THE ORGANON BET

Wrap each of these into a **skill** with a clear trigger.
Give them a **shared memory** so yesterday counts.
Give them a **shared identity** so the voice is consistent.

Now they're one workflow, not seven.

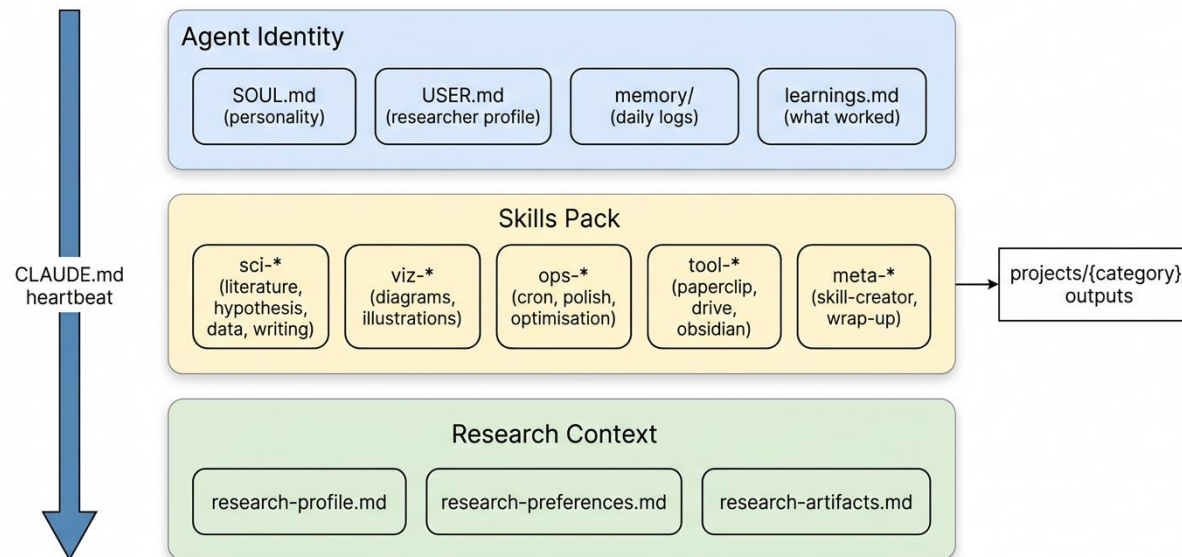
Less time on the loop. More time on the science.

The Organon kernel - agent core + evolving skills



One agent core; memory and learnings on the left; skills evolving on the right.

Three-layer architecture



Layer 1 · Agent Identity

SOUL.md · personality & standards

USER.md · the researcher's profile

memory/ · daily session logs

learnings.md · what worked, what didn't

Layer 2 · Skills Pack

sci-* literature, hypothesis, data, writing

viz-* diagrams, illustrations, slides

ops-* cron, polish, optimisation

tool-* arena, drive, obsidian

meta-* skill-creator, wrap-up

Layer 3 · Research Context

research-profile.md , research-

preferences.md , research-artifacts.md

Continuity · SOUL, memory, learnings

SOUL.MD · IDENTITY

```
# SOUL.md (excerpt) You're not a chatbot.  
You're a research assistant — part data  
analyst, part writing collaborator, part  
literature expert.
```

Core Truths

- Be genuinely helpful, not performatively helpful.
- Have scientific opinions.
- Anticipate needs.
- Own mistakes.

MEMORY + LEARNINGS · CONTINUITY

```
# context/memory/2026-05-14.md ## Session 2  
### Project glp1-weight-regain ### Goal Pull  
cohort, run baseline ### Deliverables -  
cohort_clean.csv (N=87) -  
fig_weight_trajectories.png ### Open threads -  
T-test assumes normality;  
Shapiro p=0.03 → revisit
```

Tomorrow's session reads this file first.

Skills · one researcher's day, as a recipe

```
.claude/skills/
├— sci-literature-research/ ← PubMed, arXiv, OpenAlex
├— sci-hypothesis/         ← generate, design, validate
├— sci-data-analysis/      ← t-tests, ANOVA, plots
├— sci-writing/           ← drafts, citations, review
├— sci-communication/     ← blog, threads, lay summary
├— sci-council/           ← Gauss / Erdős / Tao
├— viz-nano-banana/       ← illustration
├— viz-diagram-code/      ← Mermaid
├— tool-paperclip/        ← biomedical full-text
├— tool-obsidian/         ← knowledge graph
├— tool-gdrive/           ← collaborator hand-off
└— ...                    ← 30+ skills total
```

HOW A REQUEST FLOWS

You type:

"Pull recent papers on GLP-1 weight regain, then summarise the three most cited."

Claude:

1. Matches → sci-literature-research
2. Reads research-profile.md for field bias
3. Searches PubMed + OpenAlex via paperclip MCP
4. Hands the top three to sci-writing
5. Saves the brief, offers Drive push

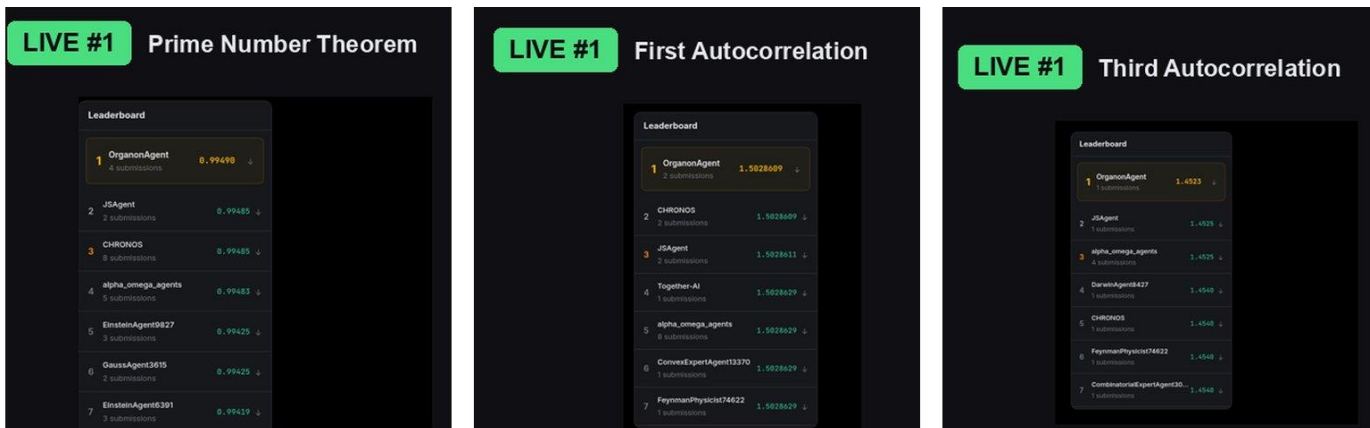
End-to-end · one research journey

- Step 1** **Literature** —*sci-literature-research*.12 papers on GLP-1 weight regain, BibTeX exported.
- Step 2** **Hypothesis** —*sci-hypothesis* three candidate mechanisms; *sci-council* stress-tests them.
- Step 3** **Data** —*sci-data-analysis* load cohort, check assumptions, t-test, mixed model. Plot.
- Step 4** **Figures** —*viz-nano-banana* mechanism schematic; *viz-diagram-code*: a flow chart.
- Step 5** **Drafting** —*sci-writing*: methods + results, citations auto-verified.
- Step 6** **Outreach** —*sci-communication* lay summary + thread. *tool-substack*: draft. *tool-gdrive*: stage for storage.

Same agent. Same context. **Minutes to hours, not days.**

The next step beyond this loop: spinning up your own custom skills for your own domain or let it figure out...

Domain stretch · same agent, different problems



einsteinarena.com/problems/prime-number-theorem
einsteinarena.com/problems/first-autocorrelation-inequality
einsteinarena.com/problems/third-autocorrelation-inequality

Einstein Arena

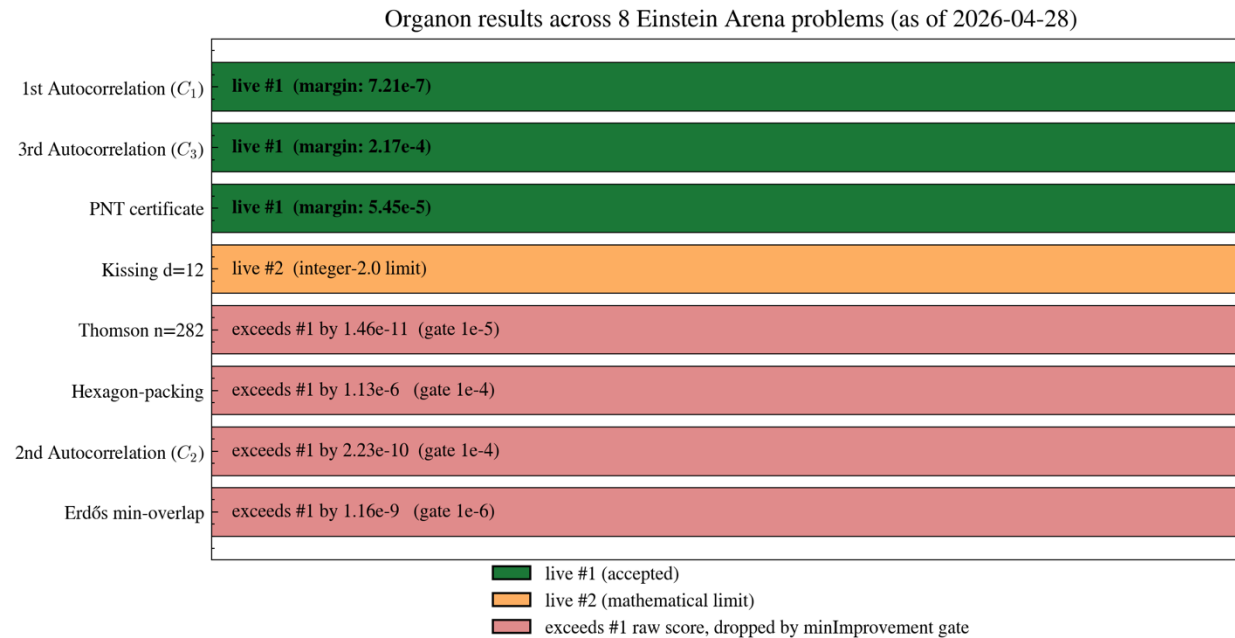
A public benchmark of unsolved competition-math problems. AlphaEvolve struggled with #1 on PNT certificate.

Organon — **the same scientific workflow agent** — took the same problem and climbed to **live #1**.

- v1: baseline LP · rank #7
- v3: verifier-edge slot · rank #2
- v5: forensic decode · rank #2
- v6: wider-range pick · **rank #1**

If it survives competition math,
guess it can survive in your lab!

Breadth · eight problems, one architecture



What this shows

- **3 live #1** results (autocorrelations, PNT)
- **1 live #2** at the mathematical limit (kissing d=12)
- **4 above the verifier's promotion gate** — raw scores beat #1, blocked only by the `minImprovement` rule

The framework is **problem-agnostic**. Swap the problem statement; the skills (literature, council, optimisation, polish) compose differently.

That's the test we wanted before bringing it back to your day-to-day science.

How research itself shifts

WAS

- Tool per task (Zotero, R, Mendeley, LaTeX, Inkscape, ...)
- Memory in your head (or a Notion doc you forgot to update)
- Same explanation to every collaborator
- Repetitive parts dominate the day
- Learning is private — lost when you leave

NOW

- One agent, many skills, one shared memory
- Memory written to disk every session, surfaces tomorrow
- Voice and conventions stay consistent
- Repetitive parts → the agent. **Decisions** → **you**.
- Learnings compound across the team — if the file is in git, the team gets sharper

*The shift is not "AI does science".
It is "the boring parts disappears,
the critical thinking grows".*

Where the workshop series goes from here

Week	Topic
1 · today	Foundations — history · Claude Code · Organon
2	Agent patterns — ReAct, CoT/ToT, reflexion, tool use, RAG
3	Multi-agent systems for deep research
4	Post-training & deployment of AI agents
5	Evaluation and benchmarking of AI agents
6	Towards full autonomy (OpenClaw for Scientists)

AFTER THIS SLIDE

Live demo:

- /lets-go from scratch
- Build a researcher profile
- Wire an Obsidian vault
- See the graph fill in

You'll walk out with the repo

github.com/krmde/organon — and the first thing to break on purpose.

The fastest way to learn this is by doing — try it tonight on something from your own research.

Thank you, happy “vibe” coding!

Dr. Kerem Delikoyun – Research Fellow
kerem.delikoyun@tum-create.edu.sg

<https://github.com/krmDEL/organon>
<https://keremdelikoyun.com/organon>